DATA37711 Homework 1

Linyan Li

Question 1

Suppose Victor has d dollars of total wealth and believes that

$$P(\text{Biden wins}) = 0.85, \qquad P(\text{Biden loses}) = 0.15.$$

The betting market prices a contract that pays \$1 if Biden wins at \$0.60.

Let $b \in [0, d]$ denote the amount of money Victor bets on Biden. Betting b dollars purchases

$$\frac{b}{0.6}$$

contracts.

Therefore, Victor's final wealth is

$$W = \begin{cases} d - b + \dfrac{b}{0.6} = d + \dfrac{2}{3}b, & \text{if Biden wins,} \\ d - b, & \text{if Biden loses.} \end{cases}$$

(a)

Suppose Victor's utility function is linear:

$$U(y) = y.$$

His expected utility from betting b dollars is

$$\begin{aligned} \mathbb{E}[U(W)] &= 0.85\left(d + \frac{2}{3}b\right) + 0.15(d - b) \\ &= 0.85d + 0.85\left(\frac{2}{3}\right)b + 0.15d - 0.15b \\ &= d + \left(0.85 \cdot \frac{2}{3} - 0.15\right)b \\ &= d + \frac{5}{12}b. \end{aligned}$$

Differentiating with respect to b,

$$\frac{d}{db}\mathbb{E}[U(W)] = \frac{5}{12} > 0.$$

Thus expected utility is strictly increasing in b. Since

$$0 \leq b \leq d,$$

the expected utility is maximized by choosing the largest possible bet:

$$b^* = d.$$

Therefore, Victor will bet his entire bankroll on Biden winning.

This illustrates a limitation of linear utility. A linear utility function assumes that the marginal utility of money is constant, so gaining an additional dollar has the same utility regardless of one's current wealth. In practice, people are generally risk-averse and the marginal utility of wealth tends to decrease as wealth increases. Therefore, a concave utility function is usually a more realistic model.

(b)

Now suppose Victor's utility function is

$$U(y) = \log(y).$$

His expected utility from betting b dollars is

$$EU(b) = 0.85 \log\left(d + \frac{2}{3}b\right) + 0.15 \log(d - b).$$

To find the optimal bet, differentiate with respect to b:

$$EU'(b) = \frac{0.85(2/3)}{d + \frac{2}{3}b} - \frac{0.15}{d - b}.$$

At an interior optimum,

$$EU'(b) = 0.$$

Therefore,

$$\frac{0.85(2/3)}{d + \frac{2}{3}b} = \frac{0.15}{d - b}.$$

Using

$$0.85 = \frac{17}{20}, \qquad 0.15 = \frac{3}{20},$$

I obtain

$$\frac{17/30}{d + \frac{2}{3}b} = \frac{3/20}{d - b}.$$

Cross-multiplying gives

$$\frac{17}{30}(d - b) = \frac{3}{20}\left(d + \frac{2}{3}b\right).$$

Multiplying both sides by 60,

$$34(d - b) = 9\left(d + \frac{2}{3}b\right).$$

Expanding,

$$34d - 34b = 9d + 6b.$$

Thus,

$$25d = 40b,$$

so

$$b^* = \frac{5}{8}d.$$

Hence Victor should bet

$$62.5\%$$

of his total wealth on Biden winning.

To verify that this is a maximum, note that

$$EU''(b) = -\frac{0.85(2/3)^2}{\left(d + \frac{2}{3}b\right)^2} - \frac{0.15}{(d - b)^2}.$$

For $0 \leq b < d$,

$$EU''(b) < 0.$$

Therefore, the expected utility is strictly concave in b, and $b^* = \frac{5}{8}d$ is the unique maximum.

(c)

Suppose the total net worth is

$$d = \$500{,}000.$$

Using the optimal betting fraction from part (b),

$$b^* = \frac{5}{8}d = \frac{5}{8}(500{,}000).$$

Therefore,

$$b^* = \$312{,}500.$$

If Biden wins, the resulting wealth would be

$$500{,}000 + \frac{2}{3}(312{,}500) = \$708{,}333.33.$$

If Biden loses, the resulting wealth would be

$$500{,}000 - 312{,}500 = \$187{,}500.$$

I would personally not make a bet of this size. Although the log-utility model implies that betting \$312,500 maximizes expected utility under the assumed probabilities, the calculation treats

$$P(\text{Biden wins}) = 0.85$$

as known and correctly specified. In reality, this probability estimate is itself uncertain. A moderate error in the estimated probability could substantially change the optimal bet, so I would prefer a much smaller position.

Question 2

Suppose that Y is binary, so that

$$Y \in \{0, 1\}.$$

I want to show that the cross-entropy loss is a proper scoring rule. That is, I want to show that the predictive function minimizing the expected loss satisfies

$$f^*(x) = P(Y = 1 \mid X = x).$$

For binary Y, the cross-entropy loss is

$$L(y, q) = -y \log q - (1 - y) \log(1 - q),$$

where $q \in [0, 1]$ is the predicted probability that $Y = 1$.

Fix $X = x$, and define

$$p = P(Y = 1 \mid X = x)$$

and

$$q = f(x).$$

Then

$$P(Y = 0 \mid X = x) = 1 - p.$$

The conditional expected loss given $X = x$ is

$$R_x(q) = \mathbb{E}\left[L(Y, q) \mid X = x\right].$$

Since Y is binary,

$$\begin{aligned} R_x(q) &= P(Y = 1 \mid X = x)L(1, q) + P(Y = 0 \mid X = x)L(0, q) \\ &= p(-\log q) + (1 - p)(-\log(1 - q)) \\ &= -p \log q - (1 - p) \log(1 - q). \end{aligned}$$

To find the value of q that minimizes the conditional expected loss, I differentiate with respect to q:

$$\frac{d}{dq} R_x(q) = -\frac{p}{q} + \frac{1 - p}{1 - q}.$$

Setting the derivative equal to zero gives

$$-\frac{p}{q} + \frac{1 - p}{1 - q} = 0.$$

Therefore,

$$\frac{p}{q} = \frac{1 - p}{1 - q}.$$

Cross-multiplying,

$$p(1 - q) = q(1 - p).$$

Expanding both sides,

$$p - pq = q - pq,$$

and hence

$$q = p.$$

To verify that this critical point is a minimum, consider the second derivative:

$$\frac{d^2}{dq^2}R_x(q) = \frac{p}{q^2} + \frac{1-p}{(1-q)^2}.$$

For $0 < p < 1$ and $0 < q < 1$,

$$\frac{d^2}{dq^2}R_x(q) > 0.$$

Thus $R_x(q)$ is strictly convex in q, and the unique minimizer is

$$q^* = p.$$

Recalling the definitions of p and q,

$$f^*(x) = P(Y = 1 \mid X = x).$$

Since this result holds for every value of x, minimizing the overall expected cross-entropy risk

$$\mathbb{E}_{X,Y}\left[L(Y, f(X))\right]$$

gives

$$f^*(x) = P(Y = 1 \mid X = x).$$

Therefore, the cross-entropy loss is a proper scoring rule for binary Y.

Question 3

For stochastic gradient descent, the condition

$$\sum_{t=1}^{\infty} \rho_t^2 < \infty$$

ensures that the effect of stochastic gradient noise eventually vanishes. If instead I use a constant step size $\rho_t = \rho > 0$, then

$$\sum_{t=1}^{\infty} \rho_t^2 = \sum_{t=1}^{\infty} \rho^2 = \infty,$$

so SGD generally does not converge exactly to the minimizer. Instead, the iterates may continue to fluctuate in a neighborhood of the optimum due to the noise in the stochastic gradients.

However, in a finite-sample setting, exact optimization is generally not necessary. Given a finite

training sample, I minimize the empirical risk

$$\widehat{R}_n(\theta) = \frac{1}{n}\sum_{i=1}^{n} L(Y_i, f_\theta(X_i)),$$

while the quantity I ultimately care about is the population risk

$$R(\theta) = \mathbb{E}_{X,Y}\left[L(Y, f_\theta(X))\right].$$

Because the training sample is finite, $\widehat{R}_n(\theta)$ is only an estimate of the population risk $R(\theta)$. Thus, even perfect optimization of the empirical risk cannot eliminate the statistical error caused by sampling variability. Therefore, it is often unnecessary to make the optimization error much smaller than this statistical error. A sufficiently small constant step size can keep SGD near the empirical minimizer while allowing faster optimization, making it reasonable in practice despite the lack of exact asymptotic convergence.

Question 4

I use the first four variables, `anchor_ratio`, `trans_range`, `node_density`, and `iterations`, to predict `ale`. The variable `sd_ale` is ignored.

The residuals are defined as

$$e_i = y_i - \hat{y}_i.$$

For each regression model, I fit the model, calculate the residuals, draw a QQ plot, and use `scipy.stats.normaltest` to test

$$H_0 : \text{the residuals are normally distributed.}$$

I use a significance level of

$$\alpha = 0.05.$$

Data Preparation

I load the dataset and use the first four variables as predictors.

```
import pandas as pd
import matplotlib.pyplot as plt

from scipy import stats
```

```
from scipy.stats import normaltest

from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.neural_network import MLPRegressor

df = pd.read_csv("mcs_ds_edited_iter_shuffled.csv")

X = df.iloc[:, :4]
y = df["ale"]
```

4.1 Linear Regression

I first fit a linear regression model and calculate the residuals.

```
linear_model = LinearRegression()
linear_model.fit(X, y)

y_pred = linear_model.predict(X)
residuals = y - y_pred

statistic, p_value = normaltest(residuals)

print("Linear Regression")
print("Normality test statistic:", statistic)
print("p-value:", p_value)

stats.probplot(residuals, dist="norm", plot=plt)
plt.title("QQ Plot - Linear Regression Residuals")
plt.savefig("linear_qq.png", dpi=300, bbox_inches="tight")
plt.show()
```

The normality test gives

$$\text{statistic} = 40.9424, \qquad p = 1.29 \times 10^{-9}.$$

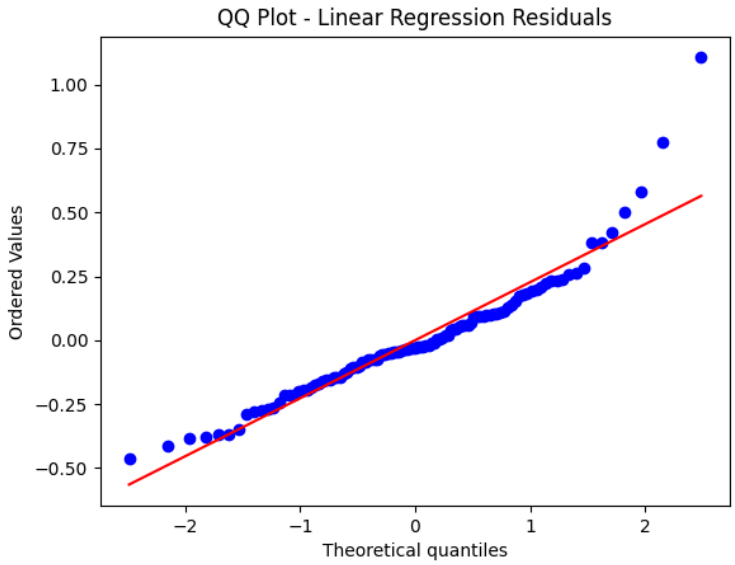


Figure 1: QQ plot of linear regression residuals.

Since

$$p < 0.05,$$

I reject the null hypothesis of normality. The QQ plot also shows noticeable deviations from the reference line, especially in the upper tail. Therefore, the linear regression residuals do not appear to be normally distributed.

4.2 Support Vector Regression

Next, I fit a support vector regression model.

```
svr_model = SVR()
svr_model.fit(X, y)

y_pred_svr = svr_model.predict(X)
residuals_svr = y - y_pred_svr

statistic_svr, p_value_svr = normaltest(residuals_svr)

print("Support Vector Regression")
print("Normality test statistic:", statistic_svr)
print("p-value:", p_value_svr)

stats.probplot(residuals_svr, dist="norm", plot=plt)
plt.title("QQ Plot - SVR Residuals")
```

```
plt.savefig("svr_qq.png", dpi=300, bbox_inches="tight")
plt.show()
```

The normality test gives

$$\text{statistic} = 54.3028, \qquad p = 1.62 \times 10^{-12}.$$

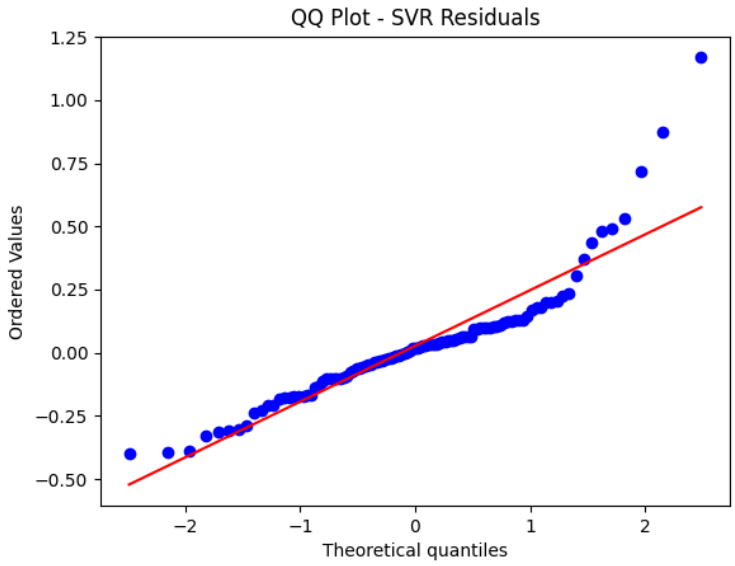


Figure 2: QQ plot of SVR residuals.

Since

$$p < 0.05,$$

I reject the null hypothesis. The QQ plot also shows substantial deviation from the normal reference line, particularly in the upper tail. Thus, the SVR residuals are not normally distributed.

4.3 Decision Tree Regression

I then fit an unrestricted decision tree regression model.

```
tree_model = DecisionTreeRegressor(max_depth=5, random_state=42)
tree_model.fit(X, y)

y_pred_tree = tree_model.predict(X)
residuals_tree = y - y_pred_tree

statistic_tree, p_value_tree = normaltest(residuals_tree)
```

```
8
9  print("Decision Tree Regression")
10 print("Normality test statistic:", statistic_tree)
11 print("p-value:", p_value_tree)
12
13 stats.probplot(residuals_tree, dist="norm", plot=plt)
14 plt.title("QQ Plot - Decision Tree Residuals")
15 plt.savefig("tree_qq.png", dpi=300, bbox_inches="tight")
16 plt.show()
```

The output of the normality test is

$$\text{statistic} = 1.3830, \qquad p = 0.5008.$$

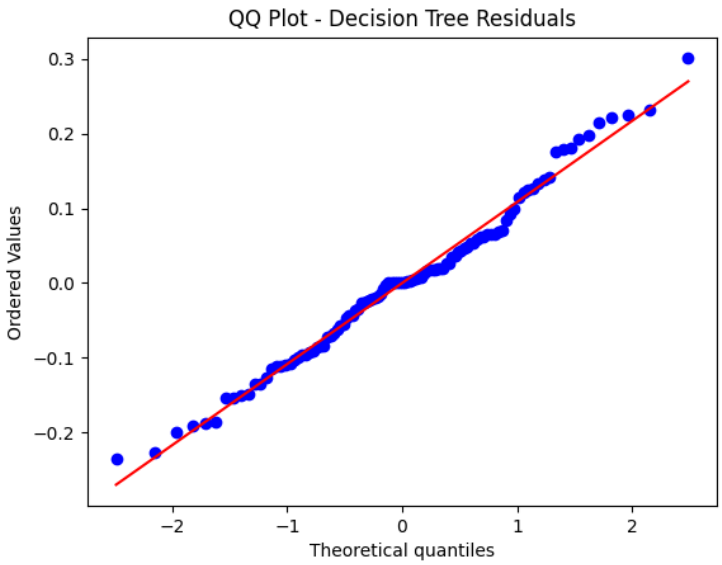


Figure 3: QQ plot of decision tree residuals.

Since

$$p > 0.05,$$

I fail to reject the null hypothesis of normality. The QQ plot also shows that the residuals lie approximately along the reference line.

4.4 MLP Neural Network

Finally, I fit a multilayer perceptron (MLP) neural network.

```python
mlp_model = MLPRegressor(
    hidden_layer_sizes=(50,),
    max_iter=5000,
    random_state=42
)

mlp_model.fit(X, y)

y_pred_mlp = mlp_model.predict(X)
residuals_mlp = y - y_pred_mlp

statistic_mlp, p_value_mlp = normaltest(residuals_mlp)

print("MLP Neural Network")
print("Normality test statistic:", statistic_mlp)
print("p-value:", p_value_mlp)

stats.probplot(residuals_mlp, dist="norm", plot=plt)
plt.title("QQ Plot - MLP Neural Network Residuals")
plt.savefig("mlp_qq.png", dpi=300, bbox_inches="tight")
plt.show()
```

The normality test gives

$$\text{statistic} = 7.4832, \qquad p = 0.0237.$$

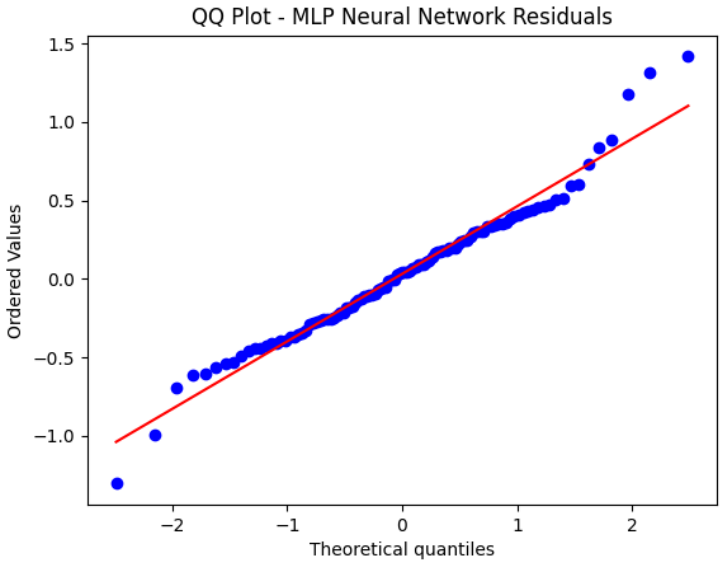


Figure 4: QQ plot of MLP neural network residuals.

The QQ plot shows that most of the residuals are relatively close to the normal reference line, although there are deviations in the tails. However,

$$p = 0.0237 < 0.05,$$

so I still reject the null hypothesis of normality at the 5% significance level.

Summary

The normality test results are summarized below.

Model	**Statistic**	p**-value**	**Conclusion**
Linear Regression	40.9424	1.29×10^{-9}	Reject H_0
SVR	54.3028	1.62×10^{-12}	Reject H_0
Decision Tree	1.3830	0.5008	Fail to reject H_0
MLP Neural Network	7.4832	0.0237	Reject H_0

Table 1: Normality test results for the residuals of the four regression models.

The linear regression, SVR, and MLP models reject the null hypothesis of normality at the 5% significance level. In contrast, the decision tree model has a p-value of 0.5008, so I fail to reject the null hypothesis of normality.

Question 5: Classification

I use the Breast Cancer Wisconsin dataset and split the data into 80% training and 20% test sets. I standardize the features before training.

Baseline

I first fit sklearn's logistic regression model.

```
data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

baseline = LogisticRegression(max_iter=5000)
baseline.fit(X_train, y_train)

y_pred = baseline.predict(X_test)
baseline_acc = accuracy_score(y_test, y_pred)
```

The test accuracy is

$$0.9737 \approx 97.37\%.$$

(a) Logistic Regression with Automatic Differentiation

I implement

$$f(x_i) = \sigma(w^T x_i + b)$$

in PyTorch and minimize the binary cross-entropy loss using SGD.

```
X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32)
X_test_t = torch.tensor(X_test, dtype=torch.float32)
y_test_t = torch.tensor(y_test, dtype=torch.float32)

w = torch.zeros(X_train.shape[1], requires_grad=True)
b = torch.zeros(1, requires_grad=True)

optimizer = torch.optim.SGD([w, b], lr=0.01)

for epoch in range(5000):
    optimizer.zero_grad()

    prob = torch.sigmoid(X_train_t @ w + b)
    prob = torch.clamp(prob, 1e-7, 1 - 1e-7)

    loss = -torch.mean(
        y_train_t * torch.log(prob)
        + (1-y_train_t) * torch.log(1-prob)
    )

    loss.backward()
    optimizer.step()
```

```
24
25 with torch.no_grad():
26     test_prob = torch.sigmoid(X_test_t @ w + b)
27     test_pred = (test_prob >= 0.5).float()
28     accuracy = (test_pred == y_test_t).float().mean()
```

I classify an observation as class 1 when its predicted probability is at least 0.5. The test accuracy is

$$0.9825 \approx 98.25\%.$$

Thus, the custom logistic regression performs slightly better than the sklearn baseline on this test split.

(b) Probit Regression

For probit regression, I replace the sigmoid function with the standard normal CDF:

$$f(x_i) = \Phi(w^T x_i + b).$$

I use the same binary cross-entropy loss and SGD procedure.

```
1  w_probit = torch.zeros(X_train.shape[1], requires_grad=True)
2  b_probit = torch.zeros(1, requires_grad=True)
3
4  optimizer_probit = torch.optim.SGD(
5      [w_probit, b_probit], lr=0.01
6  )
7
8  normal = torch.distributions.Normal(0, 1)
9
10 for epoch in range(5000):
11     optimizer_probit.zero_grad()
12
13     z = X_train_t @ w_probit + b_probit
14     prob = normal.cdf(z)
15     prob = torch.clamp(prob, 1e-7, 1 - 1e-7)
16
17     loss = -torch.mean(
18         y_train_t * torch.log(prob)
19         + (1-y_train_t) * torch.log(1-prob)
20     )
21
22     loss.backward()
```

```
    optimizer_probit.step()

with torch.no_grad():
    z_test = X_test_t @ w_probit + b_probit
    test_prob = normal.cdf(z_test)
    test_pred = (test_prob >= 0.5).float()
    accuracy_probit = (test_pred == y_test_t).float().mean()
```

The test accuracy is

$$0.9825 \approx 98.25\%.$$

The probit model achieves the same test accuracy as the custom logistic regression and slightly higher accuracy than the sklearn baseline.

Conclusion

The test accuracies are:

Model	Accuracy
sklearn Logistic Regression	97.37%
Custom Logistic Regression	98.25%
Probit Regression	98.25%

Both custom models perform similarly and achieve slightly higher test accuracy than the baseline on this train-test split.

Question 6: Regression

I use the ALE dataset and split the data into 80% training and 20% test sets. I use the first four variables as predictors and standardize the features before training. Model performance is evaluated using mean absolute error (MAE).

Baseline

I first fit sklearn's linear regression model.

```
df = pd.read_csv("mcs_ds_edited_iter_shuffled.csv")

X = df.iloc[:, :4].values
y = df["ale"].values

```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

baseline = LinearRegression()
baseline.fit(X_train, y_train)

y_pred = baseline.predict(X_test)
baseline_mae = mean_absolute_error(y_test, y_pred)
```

The test MAE is

$$0.19539.$$

(a) Linear Regression with Automatic Differentiation

I implement the linear regression model

$$f(x_i) = w^T x_i + b$$

in PyTorch and minimize the mean squared error using SGD.

```
X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32)
X_test_t = torch.tensor(X_test, dtype=torch.float32)
y_test_t = torch.tensor(y_test, dtype=torch.float32)

w = torch.zeros(X_train.shape[1], requires_grad=True)
b = torch.zeros(1, requires_grad=True)

optimizer = torch.optim.SGD([w, b], lr=0.01)

for epoch in range(5000):
    optimizer.zero_grad()

    y_pred_train = X_train_t @ w + b
    loss = torch.mean((y_train_t - y_pred_train) ** 2)

    loss.backward()
    optimizer.step()

```

```
with torch.no_grad():
    y_pred_test = X_test_t @ w + b
    mae = torch.mean(
        torch.abs(y_test_t - y_pred_test)
    )
```

The test MAE is

$$0.19539.$$

This is nearly identical to the sklearn baseline, showing that the SGD implementation converges to approximately the same linear regression solution.

(b) Student-t Loss

I next use a Student-t distribution with

$$\nu = 5$$

for the residuals. Instead of squared error, I minimize the negative log-likelihood

$$L(y_i, f(x_i)) = -\log p(y_i - f(x_i), 5).$$

```
w_t = torch.zeros(X_train.shape[1], requires_grad=True)
b_t = torch.zeros(1, requires_grad=True)

optimizer_t = torch.optim.SGD([w_t, b_t], lr=0.01)

for epoch in range(5000):
    optimizer_t.zero_grad()

    y_pred_t = X_train_t @ w_t + b_t
    residuals = y_train_t - y_pred_t

    dist = torch.distributions.StudentT(df=5.0)
    loss_t = -dist.log_prob(residuals).mean()

    loss_t.backward()
    optimizer_t.step()

with torch.no_grad():
    y_pred_test_t = X_test_t @ w_t + b_t
    mae_t = torch.mean(
        torch.abs(y_test_t - y_pred_test_t)
    )
```

The test MAE is

$$0.19410.$$

The Student-t model has a slightly lower MAE than the baseline. Because the Student-t distribution has heavier tails, its loss is less sensitive to large residuals and can be more robust to outliers.

Conclusion

The test MAEs are

Model	MAE
sklearn Linear Regression	0.19539
Custom Linear Regression	0.19539
Student-t Regression	0.19410

The custom MSE model closely matches the sklearn baseline, while the Student-t model achieves a slightly lower test MAE. Therefore, the Student-t loss gives the best performance among the three models on this train-test split.